IE3142 · DEVOPS SECURITY

Containerization, Orchestration & Secure CI/CD

Lecture 05 — Docker, Kubernetes, and integrating security into the delivery pipeline

LECTURE ROADMAP

Topics Covered

1. Introduction to Containerization
2. Containers vs Virtual Machines
3. Docker Architecture
4. Container Security Challenges
5. Container Security Best Practices
6. Container Image Security
7. Container Runtime Security
8. Introduction to Kubernetes
9. Kubernetes Architecture
10. Kubernetes Security
11. Secure Kubernetes Configuration
12. Container Security in DevSecOps

PART 1

Containerization and Orchestration Security

From Docker fundamentals to Kubernetes hardening

What is Containerization?

Containerization packages an application together with its dependencies, libraries, configuration files, and runtime components into an isolated unit called a **container.**

A container allows an application to run consistently across different environments.

"It works on my computer" → ***"It works everywhere."***

Example — packaging a web application

- Python 3.12
- Flask
- Python libraries
- Configuration files
- Application source code

↓ **Container Image**

↓ **Container**

Why Do We Use Containers?

1

Portability

Runs across laptop, test, CI/CD, cloud, and production environments.

2

Consistency

Same dependencies and runtime throughout the development lifecycle.

3

Isolation

Applications run in isolated environments without interference.

4

Scalability

Created and destroyed quickly, suited for dynamic workloads.

5

Efficiency

Share the host kernel, making them lighter than virtual machines.

6

DevSecOps Benefit

Security controls integrate into build, test, deploy, and runtime.

Containers vs Virtual Machines

Feature	Virtual Machine	Container
Virtualization	Hardware-level	OS-level
Operating System	Each VM has its own OS	Shares host kernel
Size	Generally larger	Generally smaller
Startup	Slower	Faster
Resource Usage	Higher	Lower
Isolation	Strong	Process-level isolation
Portability	Good	Very high
Typical Technology	VMware, Hyper-V	Docker, containerd

Important: Containers share the host OS kernel. a kernel or runtime vulnerability can affect multiple containers. Container isolation does not mean containers are automatically secure.

What is Container Security?

Container security is the practice of protecting containerized applications throughout their lifecycle including images, registries, runtime, hosts, networks, secrets, APIs, and orchestration platforms.



Security should not be added only after the container reaches production.

Docker Overview — Core Components

What is Docker?

Docker is a platform used to build, package, distribute, and run applications using containers.

Docker allows developers to create a consistent application environment using **Docker images**.

1

Docker Client

Used to interact with Docker.

2

Docker Engine

Runs and manages containers.

3

Docker Image

Read-only template used to create containers.

4

Docker Container

A running instance of an image.

5

Dockerfile

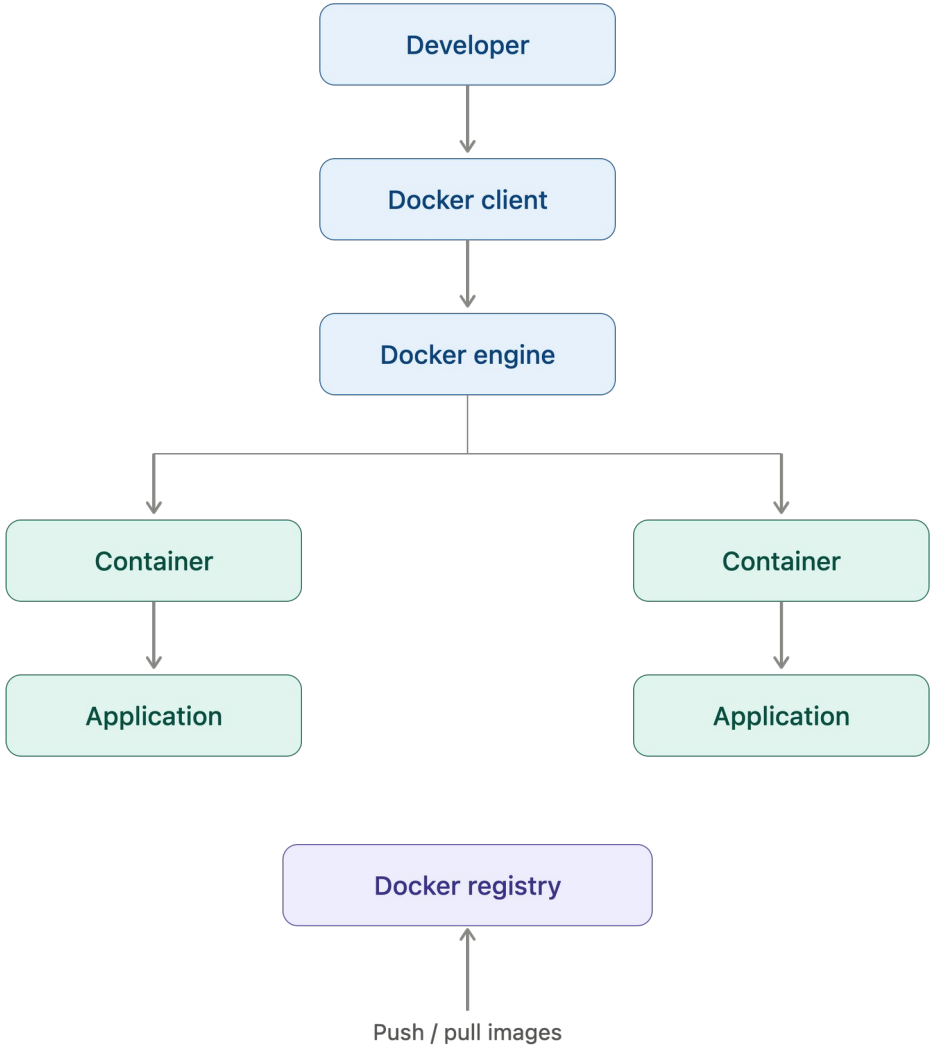
Text file with instructions for building an image.

6

Docker Registry

Repository for storing and distributing images (Docker Hub, ECR, ACR).

Docker Architecture



Security Perspective

- Compromised Docker image
- Vulnerable Docker daemon
- Insecure registry
- Excessive container privileges
- Exposed Docker API

Container Security Threats

1
Vulnerable Images
Outdated libraries or known vulnerabilities.

2
Malicious Images
Published to compromise systems.

3
Secrets in Images
Passwords, API keys, tokens accidentally included.

4
Privileged Containers
Excessive privileges can access host resources.

5
Container Escape
Breaking out to access the underlying host.

6
Insecure Docker Config
Misconfigured services expose resources.

7
Weak Networking
Poor config allows unauthorized communication.

8
Supply Chain Attacks
Compromised dependencies, images, or registries.

Container Security Best Practices

1

Image Security

Trusted, updated, minimal, scanned base images.

2

Runtime Security

No root, restrict privileges, limit resources, read-only filesystems.

3

Network Security

Segment networks, restrict comms, avoid open ports.

4

Secrets

Never hard-code — use dedicated secret management.

5

Monitoring

Monitor activity, collect logs, detect anomalies.

Principle of Least Privilege for Containers

Containers should operate with the minimum privileges required. Excessive privileges increase the impact of a compromise.

× Avoid

Container

↓ Root User

↓ Full Host-Level Privileges

✓ Prefer

Container

↓ Non-Root User

↓ Limited Permissions

↓ Required Resources Only

Best Practices

- Run applications as non-root users
- Avoid privileged containers
- Use read-only filesystems where possible
- Drop unnecessary Linux capabilities
- Restrict access to host resources

What is a Container Image?

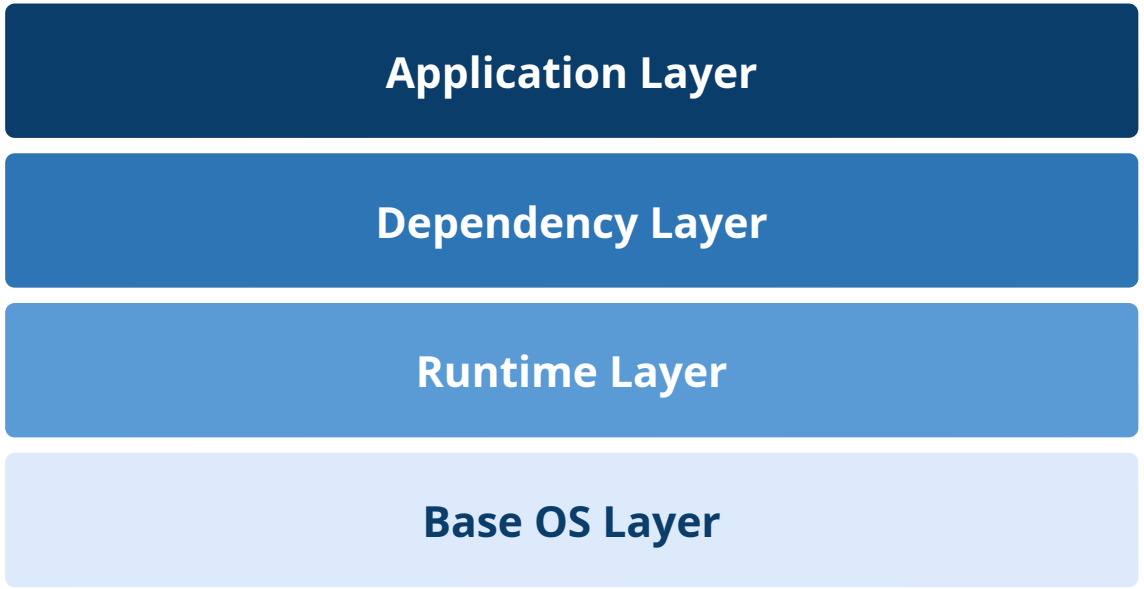
A container image is a packaged, immutable template; Image may application code, runtime, libraries, dependencies, configuration, and OS components.

Images are commonly built using a **Dockerfile**.



Container Image Layers

Container images are built using multiple stacked layers. Each Dockerfile instruction may create a new layer.



Security Importance

If an insecure package exists in an underlying layer, it may introduce vulnerabilities into the final image. Therefore:

- Use trusted base images
- Keep dependencies updated
- Remove unnecessary components
- Regularly scan images

Secure Base Images

A base image provides the starting environment for a container. The security of the final container depends partly on the security of the base image.

```
FROM python:3.12
```

Best Practices

- Use official or trusted images
- Use minimal base images where appropriate
- Avoid outdated images
- Regularly update base images
- Verify image provenance
- Scan base images for vulnerabilities

Bloated vs. Minimal Images

Large OS Image + Compiler + Package Manager + Unused Tools + Application

↓ **prefer**

Minimal image containing only what the application requires

Vulnerability Scanning of Container Images

Container image scanning identifies known vulnerabilities in OS packages, libraries, application dependencies, and software components.



Common Tools

- Trivy
- Docker Scout
- Grype
- Clair

Example Finding

```
Package:   OpenSSL
Version:   X.X.X
Severity:   HIGH
Fix Available:   Yes
```

Software Bill of Materials (SBOM)

An SBOM is an inventory of the software components and dependencies contained within an application or container.

An SBOM can identify:

- Libraries
- Versions
- Licenses
- Packages
- Dependencies
- Known vulnerabilities

Why it matters: if a critical vulnerability is found in a popular library, teams can use the SBOM to instantly check whether their containers are affected.

Example — Application SBOM

Application

```
├── Flask
├── OpenSSL
├── SQLite
├── Requests
├── NumPy
```

SBOMs improve software supply-chain visibility.

Container Registry Security

Registry Security Risks

- Unauthorized image uploads
- Malicious images
- Image tampering
- Public exposure
- Stolen credentials
- Vulnerable images

Security Best Practices

- Use private registries for sensitive apps
- Implement authentication
- Apply role-based access
- Scan images before deployment
- Sign and verify images
- Remove obsolete images
- Monitor registry activity

Container Image Signing and Verification

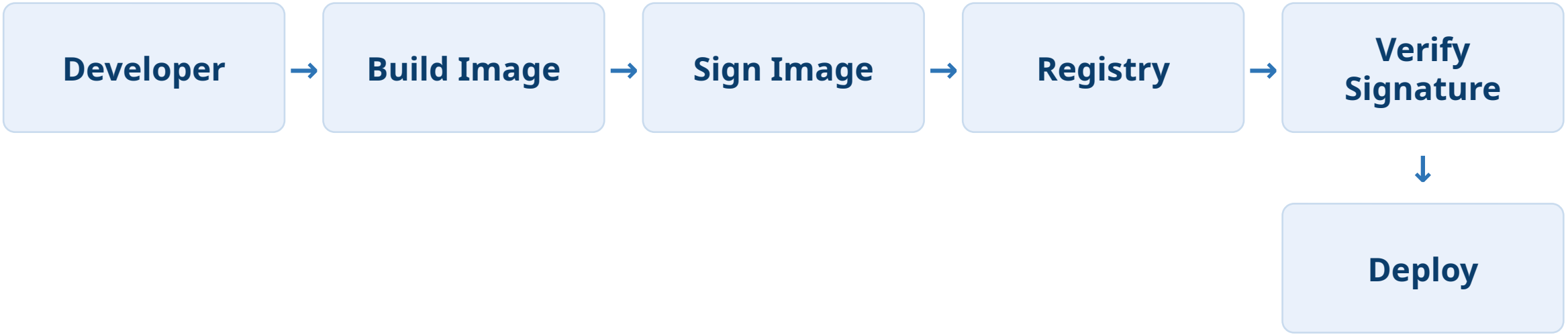


Image signing verifies a container's origin and integrity, it protects against supply-chain attacks and unauthorized modification.

Container Secrets Management

Insecure Approach

- Never place secrets in a Dockerfile
- Source Code
- Container Image
- Git Repository

Example: ENV DB_PASSWORD=MyPassword123
This can expose the password through image layers.

Secure Approach

- Docker secrets
- Kubernetes Secrets
- AWS Secrets Manager
- Azure Key Vault
- HashiCorp Vault

Container Runtime Security

1

User Security

Run as non-root; restrict privileges.

2

Filesystem Security

Read-only filesystems; restrict writable directories.

3

Resource Security

Apply CPU and memory limits to prevent exhaustion.

4

Network Security

Restrict unnecessary connections and exposed ports.

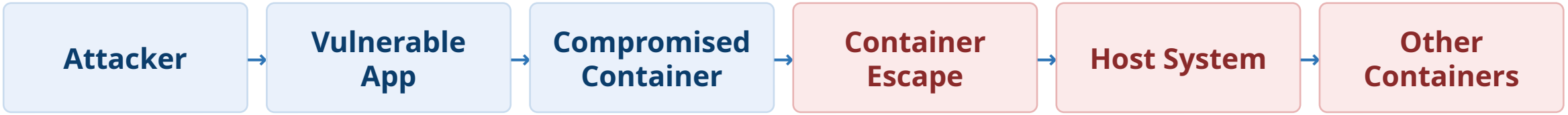
5

Monitoring

Processes, network, file access, system calls, behavior.

Container Escape

Container escape occurs when an attacker breaks out of a container's isolation and gains access to the underlying host or other resources.



Possible Causes

- Kernel vulnerabilities
- Container runtime vulnerabilities
- Excessive privileges
- Privileged containers
- Insecure host configuration

Prevention

- Keep host OS updated
- Keep container runtime updated
- Avoid privileged containers
- Use least privilege
- Restrict Linux capabilities
- Monitor runtime behavior

Resource Limits and Denial of Service

Containers share host resources. A compromised or malfunctioning container could consume excessive CPU, memory, disk space, or network bandwidth affecting other applications on the same host.

CPU Limit

Memory Limit

Storage Limit

Network Limit

Resource Limits Help Prevent

- Resource exhaustion
- Application crashes
- Denial-of-Service conditions
- Noisy-neighbor problems

Container Network Security

Security Risks

- Unnecessary open ports
- Unauthorized container communication
- Network sniffing
- Lateral movement

Best Practices

- Expose only required ports
- Restrict container-to-container communication
- Use network segmentation
- Encrypt sensitive communication
- Apply network policies
- Avoid unnecessary public exposure

Introduction to Container Orchestration

Managing a few containers manually may be simple. But consider an application with:

100	10	Multiple	Multiple
Containers	Servers	Networks	Services

Container orchestration automates:

- Deployment
- Scaling
- Networking
- Load balancing
- Service discovery
- Health monitoring
- Recovery

What is Kubernetes?

Kubernetes is an open-source container orchestration platform used to deploy, manage, scale, and operate containerized applications.

Kubernetes can automatically:

- Deploy containers
- Restart failed containers
- Scale applications
- Distribute workloads
- Provide service discovery
- Manage networking
- Perform rolling updates

Why Kubernetes Matters to DevSecOps

Kubernetes becomes part of the infrastructure where applications run. Its configuration, identities, workloads, networking, and secrets must also be secured.

Kubernetes Architecture

A Kubernetes cluster consists mainly of a Control Plane and Worker Nodes.

Control Plane

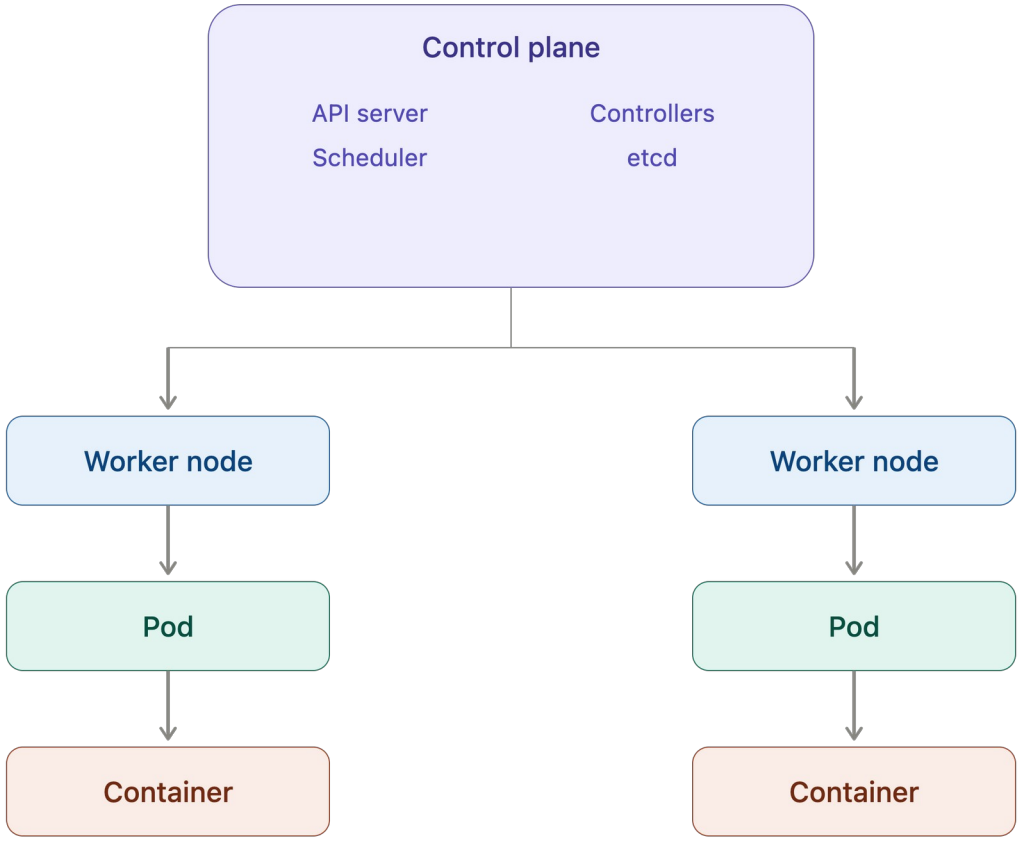
Components include:

- API Server
- Scheduler
- Controller Manager
- Etcd

Worker Nodes

Components include:

- Kubelet
- Container Runtime
- Pods



Kubernetes Security Challenges

1. Exposed API Server
2. Excessive Permissions
3. Insecure RBAC
4. Privileged Containers
5. Vulnerable Images
6. Exposed Secrets
7. Insecure Networking
8. Misconfigured Workloads
9. Compromised Nodes
10. Unrestricted Pod Comms

Kubernetes RBAC

Role-Based Access Control controls what users and workloads can do within a cluster. RBAC determines "who can access which resources?", and "what actions they can perform?".

A developer MAY be allowed to:

- View pods
- Deploy applications

But NOT:

- Delete namespaces
- Modify cluster-wide security policies
- Access secrets belonging to other apps

Best Practices

- Follow least privilege
- Avoid unnecessary cluster-admin permissions
- Separate dev and production access
- Review permissions regularly

Kubernetes Secrets

Kubernetes applications often require sensitive information such as database passwords, API keys, tokens, and certificates. Kubernetes provides a Secret object for storing this configuration.

Kubernetes Secrets should not automatically be treated as equivalent to a dedicated enterprise secret-management solution.

Organizations Should

- Restrict access using RBAC
- Encrypt secrets at rest
- Avoid exposing secrets unnecessarily
- Avoid committing secrets to source code
- Consider external secret-management systems

External Secret Managers

AWS Secrets Manager

Azure Key Vault

HashiCorp Vault

Kubernetes Network Security

Without restrictions, a compromised pod may communicate with other workloads unnecessarily. Kubernetes NetworkPolicy controls which pods, namespaces, and traffic are allowed.

Instead of allowing:

Frontend → Database

The policy allows only:

Frontend → Backend → Database

Network Policies can control:

- Which pods can communicate
- Which traffic is allowed
- Which namespaces can communicate
- Which traffic is denied

This reduces lateral movement.

Kubernetes Pod Security

Best Practices

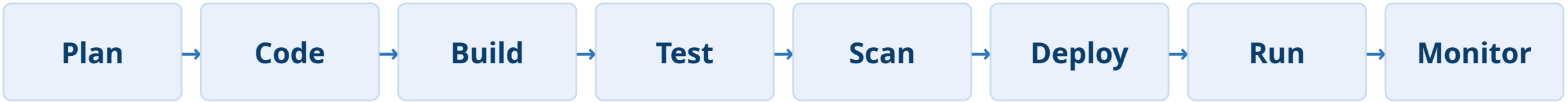
- Do not run containers as root
- Avoid privileged containers
- Drop unnecessary Linux capabilities
- Use read-only filesystems where possible
- Prevent privilege escalation
- Define resource limits
- Use appropriate security contexts

Security Context Defines

- Pods
- Containers
- Users
- Linux capabilities
- Filesystems
- Privilege escalation settings

Container Security in DevSecOps

Container security should be integrated into every stage of the DevSecOps lifecycle.



Code

Secret detection, dependency scanning

Build

Secure Dockerfile, image scanning

Registry

Image signing, access control

Deploy

Policy enforcement, config validation

Runtime

Monitoring, threat detection

This is the essence of DevSecOps for containers: security becomes continuous and automated rather than a final manual check.

PART 2

Secure Continuous Integration and Deployment

Integrating security into CI/CD pipelines, building on Lecture 01's DevSecOps foundations

What is CI/CD?

Continuous Integration (CI): frequently integrating code changes into a shared repository and automatically building and testing the application.

- Code compilation
- Unit testing
- Integration testing
- Code quality analysis
- Security scanning

Continuous Delivery (CD): prepares validated software for release.
Continuous Deployment (CD): automatically deploys validated changes to production.

Typical Pipeline



Why Integrate Security into CI/CD?

Traditional security testing happens near the end of development, vulnerabilities are discovered late, forcing teams back to earlier stages.

Traditional Approach

Code → Build → Test → Deploy

→ Security Testing

→ Vulnerability Found

DevSecOps Approach

Code → Security Checks → Build

→ Security Tests → Deploy

→ Continuous Monitoring

Benefits

- Earlier vulnerability detection
- Automated security testing
- Lower remediation cost
- Consistent security controls
- Faster feedback
- Reduced human error

Security in the CI/CD Pipeline

Stage	Security Activity
Code	SAST, secret scanning
Dependencies	SCA
Build	Secure build validation
Container	Image scanning
Testing	DAST, security tests
Infrastructure	IaC scanning
Deployment	Policy checks
Runtime	Monitoring and threat detection

Shift-Left Security in CI/CD

Shift Left means moving security activities earlier in the software development lifecycle.

Traditional — too late

Development → Testing → Deployment → Security

Shift Left

Security → Development → Testing → Deployment

Example — catching a vulnerable dependency early

Developer adds dependency → **SCA Scan** → **Vulnerability detected** → **Developer fixes it**

This reduces the cost and effort of remediation.

Static Application Security Testing (SAST)

SAST analyzes source code or compiled code without executing the application.

SAST Can Identify:

- SQL Injection
- Hardcoded credentials
- Weak cryptography
- Insecure functions
- Input validation issues

Example Tools

SonarQube · Semgrep · Checkmarx · Fortify



Software Composition Analysis (SCA)

Modern applications depend heavily on third-party libraries. SCA identifies vulnerabilities in these dependencies.

Application

```
├── Flask
├── Requests
├── OpenSSL
├── Other Libraries
```

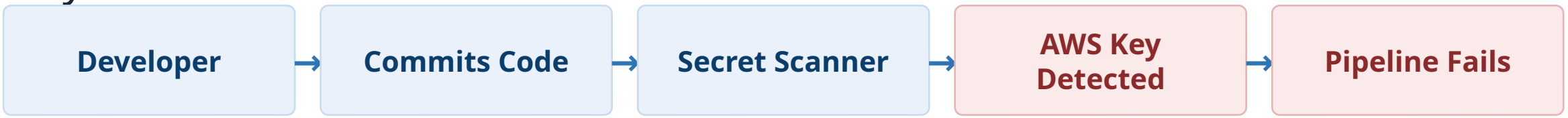
SCA Checks

- Dependency versions
- Known vulnerabilities
- Outdated packages
- License information
- Dependency relationships

Example Tools: OWASP Dependency-Check · Snyk · Dependabot

Secret Scanning

Sensitive credentials can accidentally be committed to source-code repositories; AWS access keys, API tokens, database passwords, private keys.



Best Practices

- Never hard-code secrets
- Scan repositories automatically
- Rotate exposed credentials immediately
- Use secret-management systems

Example Tools

- **GitHub Secret Scanning**
- **Gitleaks**
- **TruffleHog**

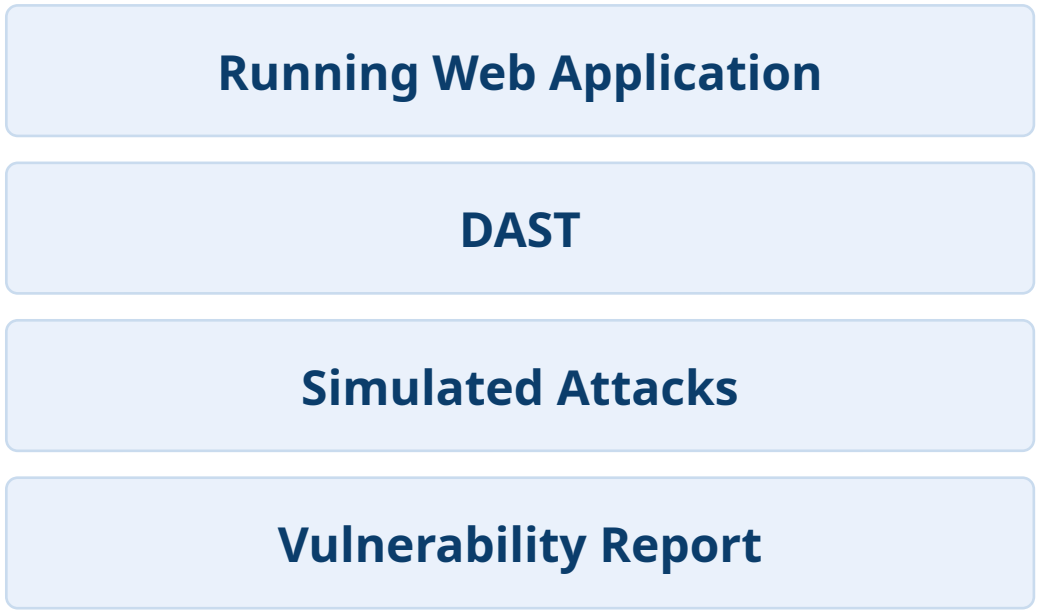
Dynamic Application Security Testing (DAST)

DAST tests a running application from the outside to identify vulnerabilities. Unlike SAST, DAST does not analyze source code directly.

DAST Can Identify:

- SQL Injection
- Cross-Site Scripting
- Authentication weaknesses
- Session management problems
- Security misconfigurations

Example Tool: OWASP ZAP



Infrastructure as Code Security

Modern DevOps environments use Infrastructure as Code (IaC) to define infrastructure through configuration files, Terraform, AWS CloudFormation, Azure Bicep.



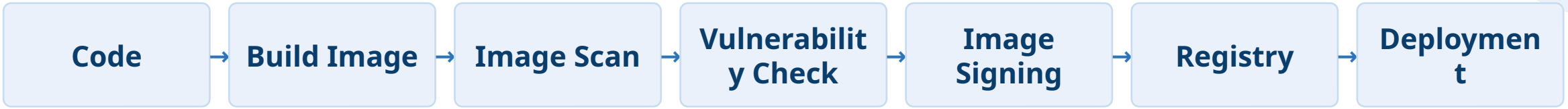
IaC Scanning Can Identify

- Public storage
- Open firewall rules
- Excessive permissions
- Unencrypted resources
- Insecure network configurations

Example Tools

- **Checkov**
- **tfsec**
- **KICS**

Container Security in CI/CD



Security Checks

- Scan image vulnerabilities
- Scan dependencies
- Detect secrets
- Verify base images
- Check Dockerfile security
- Generate SBOM
- Verify image signatures

Pipeline Decision

Security Scan

Critical Vulnerability

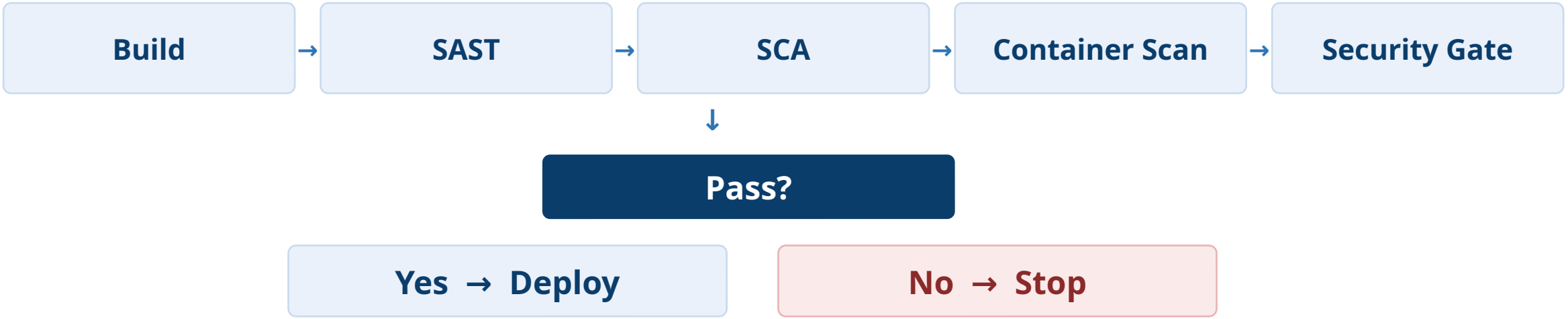
Pipeline FAILED

Deployment BLOCKED

This prevents vulnerable container images from reaching production.

Security Gates in CI/CD

A security gate is an automated condition in a CI/CD pipeline that determines whether a build can proceed to the next stage.



Example Policy

- No Critical vulnerabilities
- No exposed secrets
- Maximum 5 Medium vulnerabilities
- No high-risk IaC misconfigurations

Security Automation in CI/CD

1
Automated Code Scanning
Every code commit is scanned.

2
Automated Dependency Scanning
New dependencies are checked for vulnerabilities.

3
Automated Container Scanning
Container images are scanned before deployment.

4
Automated Infrastructure Scanning
Configurations checked before provisioning.

5
Automated Compliance
Checked against security policies.

CI/CD Pipeline Security

Pipeline Security Threats

- Compromised developer credentials
- Malicious code commits
- Compromised build servers
- Stolen CI/CD tokens
- Malicious dependencies
- Unauthorized pipeline modifications
- Supply-chain attacks

Security Controls

- Strong authentication & MFA
- Least privilege
- Protected branches
- Code review
- Secure build agents
- Secret management
- Artifact signing
- Audit logging

SUPPLY CHAIN

Software Supply Chain Security

Modern applications are built using many external components, each dependency introduces potential security risk.

Application

```
├── Open Source Libraries
├── Container Images
├── Build Tools
├── APIs
├── Third-Party Services
```

Attackers May Compromise

- Open-source packages
- Container images
- Build systems
- CI/CD pipelines
- Package repositories

Security Measures

- Dependency scanning
- Artifact verification
- SBOM
- Version pinning
- Trusted repositories
- Continuous monitoring
- Image signing

SUPPLY CHAIN

Secure Artifact Management

An artifact is a file produced during the software build process executables, packages, container images, libraries, and deployment packages.

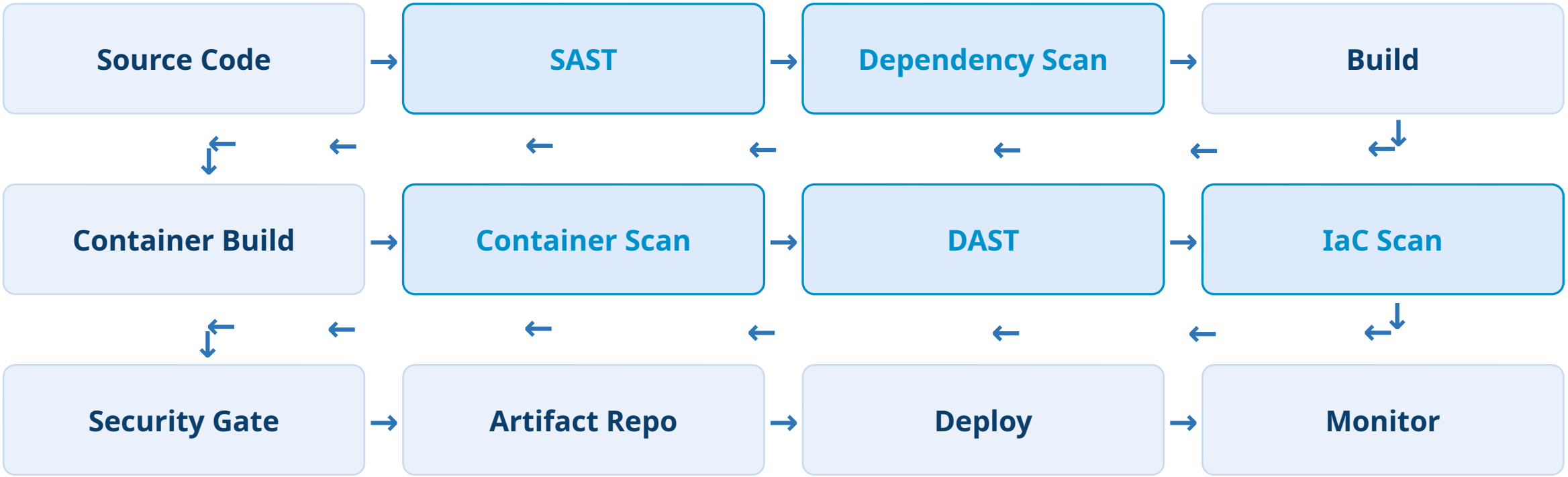
Artifacts should be securely stored and verified before deployment.

Best Practices

- Use trusted artifact repositories
- Scan artifacts
- Verify integrity
- Prevent unauthorized modification
- Control access
- Sign artifacts
- Maintain version history

SUPPLY CHAIN

DevSecOps CI/CD Pipeline



Cyan boxes are security activities. Security is continuously integrated rather than performed as a final stage.

WORKED EXAMPLE

Example: Secure CI/CD Pipeline for an Online Shopping App

1. **Code**
   Secret scanning checks for exposed credentials.

2. **SAST**
   Source code is analyzed for vulnerabilities.

3. **SCA**
   Third-party dependencies are scanned.

4. **Build**
   The application is packaged into a container.

5. **Container Scan**
   The container image is checked for vulnerabilities.

6. **IaC Scan**
   Terraform configuration is checked for misconfigurations.

7. **Security Gate**
   The pipeline checks whether security policies are satisfied.

8. **Deployment**
   Only approved artifacts are deployed.

9. **Runtime Monitoring**
   The application and infrastructure are continuously monitored.

WRAP-UP

Benefits of Secure CI/CD

1

Earlier Detection

Vulnerabilities identified earlier.

2

Lower Remediation Cost

Fix issues before deployment.

3

Faster Feedback

Automated tools provide immediate feedback.

4

Consistency

Same checks applied to every build.

5

Reduced Human Error

Automation reduces manual reliance.

6

Continuous Compliance

Security policies enforced continuously.

7

Improved Collaboration

Dev, security, and ops share responsibility.

KEY TAKEAWAYS

Container, Kubernetes & CI/CD Security

Container Security

- Portability, consistency, isolation
- Not automatically secure
- Secure images, registries, runtime, hosts
- Apply least privilege
- Scan images and dependencies
- Protect secrets · monitor runtime

Kubernetes Security

- Secure the control plane
- Apply RBAC and least privilege
- Secure pods and containers
- Use network policies
- Protect secrets
- Avoid privileged workloads

Secure CI/CD

- Integrate security throughout the pipeline
- Use SAST, SCA, DAST, IaC & container scanning
- Scan for secrets
- Use security gates
- Protect the CI/CD pipeline & supply chain
- Automate security wherever possible

DEVSECOPS

Questions?